

Step 1 — Make sure PostgreSQL is available

You need:

- PostgreSQL installed/running
- a database client such as **pgAdmin** or another SQL interface
- an empty database for the assignment

For example, you could create a database named:

movie_ratings

Once that database exists, we'll create three tables:

users
movies
ratings

The structure will be:

users
- user_id
- user_name

movies
- movie_id
- movie_title

ratings
- user_id
- movie_id
- rating

The **ratings** table connects the other two tables using IDs.

Before we write the SQL, send me the **six movies** you want to use. Then we'll build the **CREATE TABLE** statements together.

okay i have postgre running and pgadmin open, now what

Great. Since pgAdmin is already open, do this next:

1. In the left sidebar, expand **Servers** → **PostgreSQL** → **Databases**.

2. Right-click **Databases** → **Create** → **Database**.
3. Name it:

movie_ratings

4. Click **Save**.

Then click your new **movie_ratings** database, open **Tools** → **Query Tool**, and we'll create the first table.

Paste this:

```
CREATE TABLE users (  
  user_id SERIAL PRIMARY KEY,  
  user_name VARCHAR(50) NOT NULL  
);
```

Run it with the ► button.

What this does:

- **SERIAL** automatically gives each user an ID
- **PRIMARY KEY** makes that ID unique
- **VARCHAR(50)** stores the person's name
- **NOT NULL** means the name cannot be blank

Once that runs successfully, tell me what pgAdmin says, and we'll create the **movies** table next.

That error just means **R is looking in a different folder from where you saved the CSV**.

Let's find out where R is currently looking.

Step 14 — Check your working directory

Run this in R:

```
getwd()
```

It will return something like:

```
"/Users/yourname/Documents/..."
```

That is the folder where R expects to find:

movie_ratings.csv

Now you have two options. **Fastest:** move `movie_ratings.csv` into the folder shown by `getwd()`.

Or, since you're working in a QMD, put `movie_ratings.csv` in the **same project/folder as your QMD** and make sure RStudio is working from that project.

Then check whether R sees it:

```
file.exists("movie_ratings.csv")
```

We want:

```
[1] TRUE
```

Once it says `TRUE`, run:

```
movie_ratings <- read.csv("movie_ratings.csv")
```

```
head(movie_ratings)
```

Don't change anything else yet. **Get `file.exists()` to return `TRUE` and tell me what happens.**

works

Perfect. That means we've now completed the core **SQL → CSV → R dataframe** requirement.

Let's finish the R portion efficiently.

Step 15 — Inspect what came into R

Run:

```
str(movie_ratings)
summary(movie_ratings)
```

This checks that `user_name` and `movie_title` came in as text and `rating` came in as numeric/integer.

Now let's answer a simple useful question:

Which movies received the highest average ratings?

Since you're exploring methods rather than committing to `dplyr`, we can use base R:

```
aggregate(rating ~ movie_title,
```

```
data = movie_ratings,  
FUN = mean)
```

That groups the rows by movie and calculates the **mean rating for each movie**.

But there's an important problem

Our CSV only contains ratings that actually exist.

For example, Alex didn't rate every movie. Those combinations simply **aren't present in the JOIN result**.

Your assignment specifically asks you to demonstrate a reasonable strategy for **missing ratings**. We shouldn't turn an unseen movie into a 0, because that would falsely suggest the person disliked it.

Our strategy will be:

If a person did not rate a movie, represent that combination as **NA**.

That distinction matters:

Rating = 1 → person saw it and rated it poorly

Rating = NA → person didn't provide a rating

That's exactly why we shouldn't replace missing ratings with zero.

Run the `aggregate()` code first and **paste the six movie averages you get**. Then we'll explicitly create the user × movie rating matrix with **NA**s, which will satisfy the missing-data part of the assignment.

it gives me a table with the movies and their ratings

Perfect — that means the average-rating analysis works. Now let's handle the **missing ratings**, which is one of the explicit requirements.

Step 16 — Make the missing ratings visible

Right now, your dataframe only contains ratings that people actually gave. We want a table where:

- rows = users
- columns = movies
- values = ratings
- movies that someone didn't rate = **NA**

Since we're using base R, try:

```
rating_matrix <- xtabs(  
  rating ~ user_name + movie_title,  
  data = movie_ratings,  
  sparse = FALSE  
)
```

rating_matrix

However, there's something we need to investigate here: `xtabs()` can represent combinations that don't exist as **0**, which isn't what we conceptually want. A zero would look like an actual rating, even though our rating scale is only 1–5.

So instead, let's use `reshape()`:

```
rating_matrix <- reshape(  
  movie_ratings,  
  idvar = "user_name",  
  timevar = "movie_title",  
  direction = "wide"  
)
```

rating_matrix

Now the missing combinations should appear as:

NA